

Entregables Proyecto de Curso - Segundo Corte



SANTIAGO ENRIQUE BASTIDAS ESTRADA

BIBIANA SOFIA CORTES MONTILLA

JOSE ALEJANDRO RODRIGUEZ DUEÑAS

GLENN ALEXANDER WARD ANTE

Ingeniería de Software III

Profesor:

Daniel Eduardo Paz Perafan

Francisco Javier Obando Vidal

Juan Carlos Narvaez Narvaez

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Departamento de Sistemas

Ingeniería del Software III

Popayán, Mayo 2026

Historial de Revisión

Versión	Autores	Fecha
0.1	Glenn Ward, Sofia Cortes, Santiago Bastidas, Jose Rodriguez	Marzo 2026
0.2	Glenn Ward, Sofia Cortes, Santiago Bastidas, Jose Rodriguez	Abril 2026
1.0	Glenn Ward, Sofia Cortes, Santiago Bastidas, Jose Rodriguez	Abril 2026

Tabla de Contenido

	Página
1 Introducción	4
2 Descripción del problema	4
3 Metas y Restricciones de la Arquitectura	5
4 Vista de Casos de Uso y Escenarios de Calidad	8
4.1 Modelo de Casos de Uso	9
4.2 Especificación de los Casos de Uso Relevantes	10
4.3 Especificación de los Escenarios de calidad Relevantes	13
5 Vista Lógica	16
5.1 Parte Estructural: descomposición modular	16
5.1.1 Primer Nivel de Refinamiento	16
5.1.2 Segundo Nivel de Refinamiento	18
5.2 Parte Dinámica (Comportamiento)	23
5.2.1 View Packet Negocio Ventas	23
5.2.2 View Packet Negocio Producto	26
5.2.3 View Packet Negocio Envío	28
5.2.4 View Packet Negocio Financiero	30
6 Vista de Despliegue	32
6.1 Tecnología requerida	33
7 Rationale	

1. Introducción

Este documento describe la arquitectura del sistema de software de Piedra azul. Su objetivo es que sirva como referencia técnica oficial para el equipo de desarrollo. Este documento se limita a explicar como esta construido el sistema y a explicar por qué se tomaron las decisiones arquitectónicas fundamentales para el diseño e implementación del sistema, incluyendo las vistas estructurales, de comportamiento y de despliegue que permiten satisfacer los requisitos funcionales y de calidad identificados., además de explicar qué implicaciones técnicas trae cada decisión en el ciclo de vida del proyecto.

Audiencia objetivo: Este documento va dirigido a público técnico, como desarrolladores, arquitectos de software, diseñadores, que esté interesada en entender la arquitectura del sistema.

2. Descripción del problema

Contexto y antecedentes:

La Red de Servicios Médicos de Piedrazul de Popayán, ubicada en el kilómetro 5 vía al Huila, es una organización sin ánimo de lucro que brinda atención en salud a la comunidad en general, enfocándose en tratamientos de medicina alternativa. Entre los servicios ofrecidos se encuentran la terapia neural, la quiropráctica y la fisioterapia. La atención al público se realiza de lunes a viernes en horas de la mañana y, debido a la naturaleza de los tratamientos, el centro recibe diariamente un número significativo de pacientes.

Inicialmente, Piedrazul no contaba con un sistema formal de asignación de citas médicas. Los pacientes asistían directamente al centro y aguardaban su turno de atención, lo que funcionaba adecuadamente mientras el volumen de usuarios era bajo. Sin embargo, con el paso del tiempo y el crecimiento sostenido de la demanda, este modelo se volvió ineficiente, generando largas esperas, desorganización y dificultades para planificar la jornada laboral del personal médico.

Como primera solución, se implementó una agenda de citas mediante una hoja de cálculo, la cual permitió llevar un control básico de los pacientes y los horarios. No obstante, esta alternativa pronto resultó insuficiente debido al aumento del número de citas, la duplicidad de información y la alta probabilidad de errores humanos en la gestión manual. Esta situación llevó al desarrollo de una primera versión de un software de escritorio, orientado a apoyar la gestión administrativa y operativa del centro médico.

Actualmente, el proceso de agendamiento de citas se realiza de forma manual a través de una aplicación de escritorio monolítica y desactualizada, lo que genera los siguientes problemas:

- Los pacientes no pueden agendar citas de forma autónoma; deben comunicarse por WhatsApp con un agendador.
- No existe validación automática de disponibilidad de médicos, generando conflictos de horarios.
- Los médicos no tienen acceso en tiempo real a sus citas del día.
- No hay control de acceso por roles; cualquier usuario puede ver información sensible.

El problema surge debido a la gran cantidad de citas diarias que hay actualmente y el tiempo que involucra agendar mediante la aplicación de escritorio, una por una.

Necesidades y objetivos:

Objetivo: Se requiere una aplicación web que permite agendar citas de forma autónoma de acuerdo a la configuración que se le dé al sistema de médicos disponibles, fechas, horarios y espacio entre cita y cita. De esta forma, la aplicación liberará el tiempo en las tardes de los médicos y funcionarios de Piedrazul.

1. **Gestión de usuarios del sistema:** El sistema debe permitir la creación, edición, consulta y desactivación de usuarios que interactúan con la aplicación. Para el registro de un usuario se deben solicitar, como mínimo, los siguientes datos: Nombre de usuario (login), único en el sistema, Contraseña (almacenada de forma segura), Nombre completo, Rol del usuario (Médico / Terapeuta, Agendador de citas, Administrador) y Estado del usuario (activo o inactivo).
 - Cuando el usuario tenga asignado el rol de médico/terapeuta, el sistema debe permitir asociar el usuario, con el registro del médico o terapeuta correspondiente, garantizando la trazabilidad de las acciones realizadas por dicho profesional.
 - El sistema podría permitir el cambio de contraseña y la recuperación de acceso mediante preguntas de seguridad o correo electrónico (opcional para el alcance del curso).
2. **Autenticación y control de acceso:** El sistema debe contar con un mecanismo de autenticación que permita a los usuarios iniciar sesión mediante su nombre de usuario y contraseña. Una vez autenticado, el sistema debe restringir el acceso a las funcionalidades según el rol asignado, garantizando que cada usuario solo pueda realizar las acciones autorizadas para su perfil.
 - El sistema podría registrar intentos fallidos de inicio de sesión como parte del proceso de auditoría.
3. **Gestión de médicos y terapeutas:** La aplicación debe permitir la administración de médicos y terapeutas, incluyendo las operaciones de creación, edición, consulta y cambio de estado. Para cada profesional se deben registrar los siguientes datos: Nombres completos, Tipo de profesional (médico o terapeuta), Especialidad (Terapia neural, Quiropraxia, Fisioterapia), Intervalo de atención entre citas (en minutos) y Estado (activo o inactivo)
 - El sistema solo debe permitir la asignación de citas a médicos o terapeutas que se encuentren activos. Podría contemplarse la configuración de horarios de atención por profesional (por ejemplo, días y franjas horarias disponibles).
4. **Agendamiento autónomo de citas:** El sistema debe permitir que un usuario interesado en recibir atención médica acceda al sistema web mediante su usuario y contraseña y pueda agendar una cita de manera autónoma.
 - A partir de la información configurada en el sistema (disponibilidad de profesionales, intervalos entre citas, especialidades, festivos y horarios disponibles de cada médico/terapeuta), el sistema debe calcular y sugerir la fecha y hora más cercana y adecuada para la atención.
 - El sistema podría mostrar varias opciones de horarios disponibles por médico para que el paciente seleccione la que más le convenga.
5. El sistema debe implementar mecanismos de seguridad y validación que permitan prevenir la creación de cuentas falsas y el agendamiento de citas por parte de usuarios no reales, así como el registro de información fraudulenta o inconsistente. En particular, el sistema debe contemplar:
 - Validación de la identidad del usuario durante el proceso de registro (por ejemplo, verificación de correo electrónico o número telefónico).
 - Controles que limiten el uso automatizado o malicioso del sistema para la creación masiva de cuentas o reservas.
 - Verificación básica de los datos ingresados por los usuarios, reduciendo el ingreso de información incompleta o claramente falsa.

Estos mecanismos deben diseñarse de forma que no afecten significativamente la usabilidad del sistema para los usuarios legítimos.

6. **Agendamiento manual de citas:** El sistema debe permitir que los usuarios con rol de médico/terapista o agendador de citas puedan registrar citas manualmente, replicando el funcionamiento del software de escritorio actual. Se debe tener cuidado de no hacer más de una reserva con el mismo médico para una misma fecha.
 - Durante este proceso se deben registrar los datos básicos del paciente, el profesional asignado, la fecha, la hora y el tipo de atención.
7. **Re-agendamiento de citas:** El sistema debe permitir a los usuarios con rol de médico/terapista o agendador realizar el re-agendamiento de citas existentes, manteniendo un funcionamiento similar al de la versión de escritorio
 - El sistema debe conservar el historial de cambios asociados a la cita, incluyendo la fecha y el responsable del re-agendamiento.
8. **Exportación de citas:** La aplicación debe permitir la exportación de citas médicas correspondientes a una fecha específica y a un médico o terapeuta determinado, en un formato de texto compatible con hojas de cálculo (por ejemplo, CSV).
 - El archivo exportado debe poder ser abierto, editado e impreso, facilitando la organización diaria del centro médico.
9. **Historia clínica básica:** El sistema debe permitir que el médico o terapeuta registre una historia clínica básica asociada a cada cita atendida. Para cada registro clínico se deben almacenar, como mínimo: Fecha y hora de la atención, Profesional que realizó el control, Descripción del procedimiento o control médico realizado
 - Esta información debe estar disponible únicamente para usuarios autorizados.
10. **Auditoría del sistema:** El sistema debe contar con un mecanismo de auditoría que registre las acciones relevantes realizadas por los usuarios, incluyendo, entre otras: Creación, modificación y desactivación de usuarios, Agendamiento y re-agendamiento de citas, Registro y modificación de historias clínicas
 - Para cada evento se debe almacenar quién realizó la acción y en qué fecha y hora.
11. **Reportes y estadísticas:** La aplicación debe permitir la generación de reportes estadísticos, tales como: Cantidad de citas por mes, Cantidad de citas por médico o terapeuta, Comparativos por especialidad. Los reportes deben presentarse de forma clara y comprensible, preferiblemente mediante tablas y gráficos. Los reportes podrían exportarse a formatos comunes (PDF o Excel) como funcionalidad opcional.

Problemas y desafíos actuales:

El proceso actual de agendamiento de citas en Piedrazul depende completamente del personal médico, quienes deben atender solicitudes presencialmente o por WhatsApp de lunes a viernes. Esto genera una carga operativa significativa sobre los médicos y funcionarios, quienes deben interrumpir sus actividades para registrar cada cita manualmente en el software de escritorio. El volumen creciente de pacientes ha hecho que este modelo sea insostenible, generando desorganización, largas esperas y dificultades para planificar la jornada laboral del personal médico.

Limitaciones y restricciones:

El software de escritorio actual opera de forma monolítica y no permite el autoservicio por parte de los pacientes, toda cita debe ser gestionada por un funcionario del centro. El sistema no cuenta con un canal web que permita al paciente agendar de forma autónoma, no escala ante el crecimiento del número de usuarios, y su arquitectura no facilita la incorporación de nuevas funcionalidades. Adicionalmente, el sistema solo funciona en las instalaciones físicas del centro, no tiene mecanismos de autenticación robustos y no soporta el acceso concurrente de múltiples usuarios.

Impacto del problema:

La ausencia de un canal de autogestión obliga a los médicos a dedicar tiempo de la tarde exclusivamente a recibir solicitudes de citas, reduciendo su disponibilidad para otras actividades clínicas y administrativas. Los pacientes deben comunicarse directamente con el centro para agendar, lo que genera cuellos de botella en horas de alta demanda y puede resultar en pérdida de citas por falta de respuesta oportuna. A nivel organizacional, la dependencia de un software de escritorio monolítico limita la capacidad de Piedrazul de crecer y modernizar sus procesos, afectando tanto la experiencia del paciente como la eficiencia operativa del centro médico.

3. Representación de la Arquitectura

La arquitectura del sistema sigue el estilo de Monolito Modular, donde el sistema está compuesto por un único desplegable pero organizado internamente en módulos cohesivos por dominio de negocio. Esta decisión se justifica por:

- Contexto organizacional pequeño: Piedrazul es un centro médico de tamaño reducido.
- Equipo de desarrollo pequeño: grupo de 4 estudiantes.
- Preparación para evolución: los módulos pueden extraerse a microservicios si el sistema crece.

El documento está organizado siguiendo el modelo de vistas 4+1 de Philippe Kruchten, adaptado al contexto del proyecto: vista de casos de uso, vista lógica, vista de componentes y conectores, y vista de despliegue.

4. Metas y Restricciones de la Arquitectura

- **Seguridad:** El sistema debe autenticar y autorizar a todos los usuarios mediante tokens JWT generados por KeyCloak. Cada rol (ADMINISTRADOR, AGENDADOR, MEDICO/TERAPISTA, PACIENTE) tiene acceso restringido a los recursos que le corresponden.
- **Disponibilidad:** El sistema debe estar disponible durante el horario de atención de Piedrazul con el menor tiempo de inactividad posible.
- **Usabilidad:** El agendador debe poder registrar una cita en menos de 2 minutos. El sistema muestra únicamente las franjas horarias disponibles según la configuración del médico.
- **Modificabilidad:** La arquitectura modular permite agregar nuevos módulos de dominio (facturación, historias clínicas) sin afectar los módulos existentes.
- **Rendimiento:** Las consultas de citas y franjas disponibles deben responder en el menor tiempo disponible.

4.1 Vista de Casos de Uso y Escenarios de Calidad

4.2 Modelo de Casos de Uso

El modelo de casos de uso del sistema de agendamiento de citas Piedrazul es presentado en la Figura 1. En él se pueden visualizar los actores principales del sistema, así como los principales casos de uso que estos le darán al sistema.

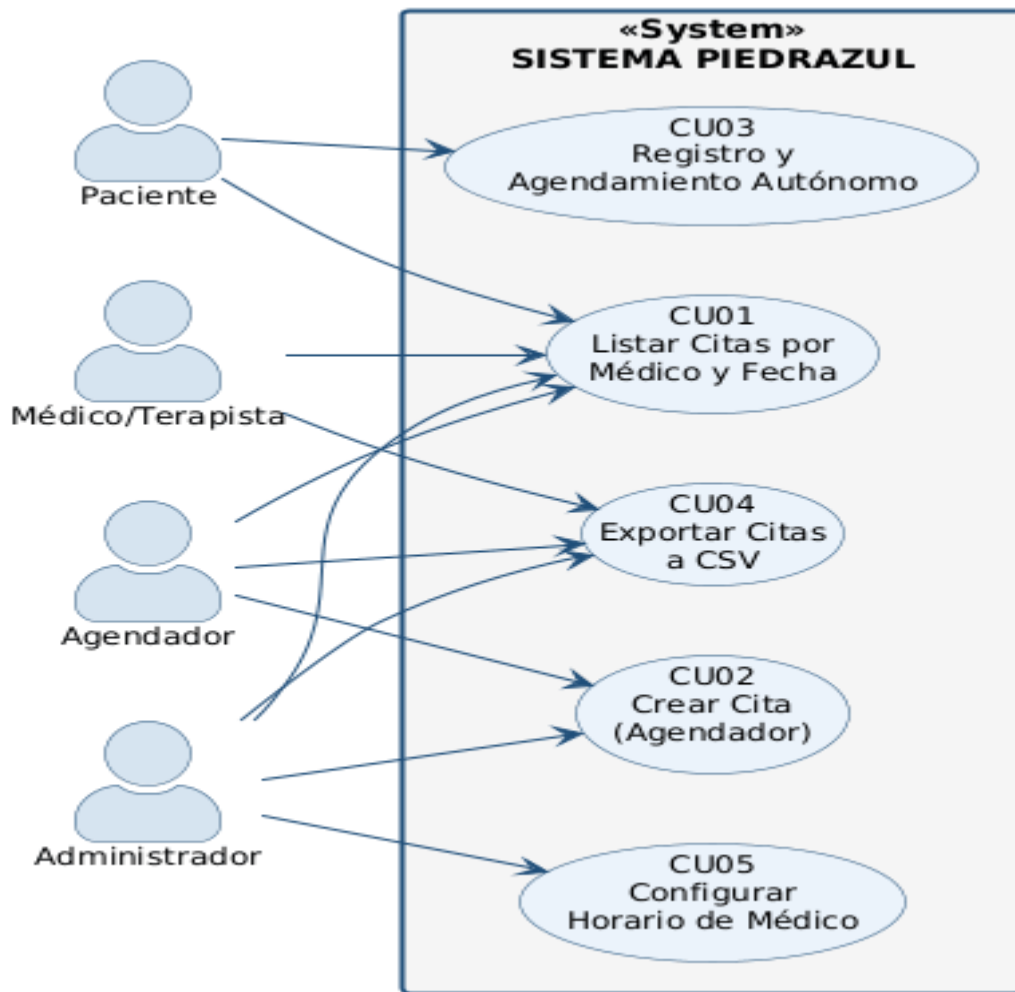


Figura 1. Modelo de casos de uso

Los actores principales del sistema se describen a continuación:

- **Paciente:** usuario que agenda citas de forma autónoma y consulta su historial de citas mediante autenticación con Keycloak.
- **Agendador:** registra pacientes y citas manualmente, reagenda y cancela citas, y exporta listados en formato CSV.
- **Médico/Terapeuta:** consulta sus citas del día y las franjas horarias disponibles según su configuración.
- **Administrador:** configura los parámetros del sistema, gestiona los médicos y tiene acceso a todas las funciones del sistema.

El modelo de casos de uso se describe de manera general partiendo de sus procesos de negocio de la siguiente forma:

El Agendador usa el sistema para gestionar citas médicas: puede Crear Cita, Reagendar Cita o Cancelar Cita. También puede consultar citas por médico y fecha mediante el caso de uso Listar Citas por Médico y Fecha, y exportar el listado diario usando el caso de uso Exportar Citas a CSV.

El Administrador puede gestionar los parámetros del sistema mediante el caso de uso Configurar Horario de Médico, que define las franjas de atención y días disponibles por médico. Además, el Administrador tiene acceso a todos los casos de uso del Agendador.

El Paciente puede registrarse de forma autónoma en el sistema y luego agendar citas mediante el caso de uso Registro y Agendamiento Autónomo. También puede consultar sus citas mediante **Listar Citas** por Médico y Fecha, pero no tiene acceso a la exportación CSV.

El Médico/Terapeuta realiza consultas sobre su agenda diaria ejecutando el caso de uso Listar Citas por Médico y Fecha y puede exportar su listado mediante Exportar Citas a CSV.

4.3 Especificación de los Casos de Uso Relevantes

Se identificaron los casos de uso más relevantes para el desarrollo de la arquitectura. Los criterios usados para dicha determinación se describen en la Tabla 1.

Caso de Uso	Criterio
CU01 -Listar Citas por Médico y Fecha	Alta frecuencia de uso. Es la operación principal del sistema durante la jornada de atención. Involucra restricciones de rol y rendimiento.
CU02 -Crear Cita (Agendador)	Representa el flujo de negocio principal RF2. Involucra validación de disponibilidad y restricciones horarias. Alto impacto en la usabilidad y seguridad.
CU03 -Registro y Autónomo (Paciente)	Punto extra RF3. Involucra autenticación con Keycloak y creación dual en H2 y Keycloak. Escenario crítico de seguridad e integridad de datos.
CU04 -Exportar Citas a CVS	RF5. Necesario para la operación diaria del consultorio. Tiene restricciones de rol estrictas (PACIENTE excluido).
CU05 -Configurar Horario de Médico	RF4. Define las franjas disponibles. Impacta directamente en CU01 y CU02. Escenario de alta cohesión funcional.

Tabla 1. Casos de Uso más relevantes y criterios de selección

ID	CU01
Nombre	Listar Citas por Médico y Fecha
Actores	Agendador, Medico/Terapeuta, Administrador
Sinopsis	El usuario consulta todas las citas confirmadas o completadas de un médico en una fecha determinada para organizar la jornada de atención.
Curso Típico de Eventos	
	1. El usuario envía GET /api/citas/medico/{id}/fecha/{fecha} con token JWT. 2. Spring Security valida el token y el rol del usuario. 3. CitaService consulta las citas del médico en esa fecha excluyendo las canceladas. 4. El sistema retorna la lista de citas con datos del paciente, hora y estado.
Extensiones	
	1a. Token inválido o ausente → 401 Unauthorized. 2a. Rol no autorizado para el endpoint → 403 Forbidden. 3a. Médico no encontrado con el id indicado → 404 Not Found.
Prioridad AQ	Alta

ID	CU02
----	------

Nombre	Crear Cita Manual por Agendador
Actores	Agendador
Sinopsis	El agendador registra una cita para un paciente con un médico y una fecha y hora específica , validando la disponibilidad automáticamente
Curso Típico de Eventos	
	5. El agendador envía POST /api/citas con medicold, pacienteld, fecha, hora y observación. 6. CitaService valida que no exista otra cita para ese médico en esa fecha y hora. 7. Se crea la cita con estado confirmada y se registra quién la creó (registradoPor). 8. El sistema retorna la cita creada con código HTTP 201.
Extensiones	
	2a. Conflicto de horario → 400 con mensaje 'El médico ya tiene una cita para esa fecha y hora'. 1a. Médico o Paciente no encontrado → 400 con mensaje descriptivo.
Prioridad AQ	Alta

ID	CU03
Nombre	Registro Autónomo por Paciente
Actores	Paciente
Sinopsis	Un paciente nuevo se registra en el sistema. El registro crea el usuario tanto en la base de datos H2 como en Keycloak , permitiendo luego iniciar sesion con sus credenciales
Curso Típico de Eventos	
	9. El paciente envía POST /api/auth/registro con sus datos personales y credenciales. 10. KeycloakAdminService crea el usuario en Keycloak asignándole el rol paciente. 11. Si Keycloak confirma la creación, se guarda el paciente en H2. 12. El sistema retorna los datos del paciente creado con código HTTP 201.
Extensiones	
	2a. Username duplicado en Keycloak → 409 Conflict. 3a. Error al guardar en H2 → rollback: se elimina el usuario de Keycloak y se retorna 500.
Prioridad AQ	Alta

ID	CU04
Nombre	Exportar Listas de Citas en Formato CSV
Actores	Agendador, Medico/Terapista,Administrador
Sinopsis	El usuario descarga un archivo CSV con las citas de un médico en una fecha, para usarlo como listado impreso durante la jornada de atención.
Curso Típico de Eventos	
	13. El usuario accede a GET /api/citas/medico/{id}/fecha/{fecha}/export/csv con token JWT. 14. El sistema genera el CSV con columnas: Orden, Hora, Nombre Paciente, Documento, Celular, Estado, Observación. 15. La respuesta incluye header Content-Disposition con nombre de

	archivo sugerido. 16. El navegador descarga el archivo automáticamente.
Extensiones	
	1a. Rol PACIENTE → 403 Forbidden (no tiene permiso para exportar). 2a. Campos con comas en los datos se envuelven en comillas para CSV válido.
Prioridad AQ	Media

ID	CU05
Nombre	Configurar Horario de Médico
Actores	Administrador
Sinopsis	El administrador define o actualiza el horario de atención de un médico terapeuta: hora inicio , hora de fin , dias de atención e intervalos de citas
Curso Típico de Eventos	
	17. El administrador envía PUT /api/medicos/{id} con los datos del horario (horaInicio, horaFin, diasAtencion, intervaloCitas). 18. El sistema valida que los campos obligatorios estén presentes y en formato válido. 19. Se actualiza el médico en la base de datos H2 con los nuevos parámetros. 20. El sistema retorna el médico actualizado con código HTTP 200.
Extensiones	
	1a. Médico no encontrado con el id indicado → 404 Not Found. 2a. Datos inválidos (ej: horaFin antes de horaInicio) → 400 Bad Request. 1b. Rol no autorizado (solo ADMINISTRADOR puede configurar) → 403 Forbidden.
Prioridad AQ	Alta

4.4 Especificación de los Escenarios de calidad Relevantes

ID	QS1
Atributo de Calidad	Seguridad
Fuente	Usuario externo no autenticado o con rol insuficiente
Estímulo	Petición HTTP a un endpoint protegido sin token JWT o con rol no autorizado
Artefacto	API Spring Boot — SecurityConfig.java
Entorno	Operación normal
Respuesta	Spring Security intercepta primero la petición antes de que llegue al controller y valida la firma RSA del token utilizando la clave del keycloak. Luego verifica los roles con el authorizeHttpRequest.
Medida	Sin el token envía una respuesta 401 (Unauthorized), si el rol es insuficiente envía una respuesta 403 (Forbidden) y si tiene los tokens RSA-256 firmados

	por Keycloak responde con los endpoints 100% protegidos (excepto /api/auth/registro).
--	---

ID	QS1
Atributo de Calidad	Seguridad
Fuente	Usuario externo no autenticado o con rol insuficiente
Estímulo	Petición HTTP a un endpoint protegido sin token JWT o con rol no autorizado
Artefacto	API Spring Boot — SecurityConfig.java
Entorno	Operación normal
Respuesta	Spring Security intercepta primero la petición antes de que llegue al controller y valida la firma RSA del token utilizando la clave del keycloak. Luego verifica los roles con el authorizeHttpRequest.
Medida	Sin el token envía una respuesta 401 (Unauthorized), si el rol es insuficiente envía una respuesta 403 (Forbidden) y si tiene los tokens RSA-256 firmados por Keycloak responde con los endpoints 100% protegidos (excepto /api/auth/registro).

ID: QS3

Aplicación: Local

5. Vista Lógica

5.1 Parte Estructural: descomposición modular

El sistema Piedrazul sigue una arquitectura de Monolito Modular. En este primer nivel de refinamiento se identifican los módulos principales del sistema, organizados por dominio de negocio. Cada módulo agrupa la funcionalidad que le corresponde y se comunica con los demás a través de interfaces.

Los módulos identificados en el primer nivel son: Autenticación, que gestiona el acceso seguro al sistema mediante Keycloak y JWT; Agendamiento, que centraliza la creación, reagendamiento y cancelación de citas médicas; Configuración, que permite al Administrador definir los parámetros del sistema (ventana de agendamiento, horarios de médicos), y los servicios que proveen acceso a base de datos y utilidades compartidas por los demás módulos.

La Tabla A describe estos subsistemas con sus atributos de calidad y casos de uso asociados:

5.1.1 Primer Nivel de Refinamiento

El sistema Piedrazul sigue una arquitectura de Monolito Modular. En este primer nivel de refinamiento se identifican los módulos principales del sistema, organizados por dominio de negocio. Cada módulo agrupa la funcionalidad cohesiva que le corresponde y se comunica con los demás a través de interfaces bien definidas.

Los módulos identificados en el primer nivel son: **Autenticación**, que gestiona el acceso seguro al sistema mediante Keycloak y JWT; **Agendamiento**, que centraliza la creación, reagendamiento y cancelación de citas médicas; **Configuración**, que permite al Administrador definir los parámetros operativos del sistema (ventana de agendamiento, horarios de médicos); y **Servicios Transversales**, que provee acceso a base de datos y utilidades compartidas por los demás módulos.

La Tabla A describe estos subsistemas con sus atributos de calidad y casos de uso asociados:

Módulo	Descripción	Atributos Escenarios	Casos de Uso
Autenticación	Gestiona la autenticación y autorización de usuarios. Delega la identidad a Keycloak y valida tokens JWT en cada petición mediante Spring Security.	Seguridad, Control de acceso por roles	CU03
Agendamiento	Gestiona la creación, reagendamiento, cancelación y consulta de citas. Valida disponibilidad de médicos y genera franjas horarias automáticamente.	Disponibilidad, Usabilidad, Rendimiento	CU01, CU02, CU03
Configuración	Exclusivo del Administrador. Permite definir la ventana de agendamiento y los horarios de atención por médico. Sus cambios impactan directamente el cálculo de franjas del módulo de Agendamiento.	Modificabilidad, Seguridad	CU02 (admin)
Servicios Transversales	Provee infraestructura compartida: acceso a la base de datos H2 (JPA/Hibernate), utilidades de autenticación y comunicación con la API de administración de Keycloak.	Todos los atributos	CU01, CU02, CU03

Tabla A. Localización de requisitos en los subsistemas del primer nivel.

5.1.2 Segundo Nivel de Refinamiento

5.1.2.1 Módulo 1: Autenticación

El módulo de autenticación ha sido descompuesto siguiendo un estilo de capas. La autenticación se delega completamente a Keycloak como servidor de identidad externo, mientras que Spring Security actúa como Resource Server OAuth2 validando los tokens JWT en cada petición entrante. Las capas identificadas son:

Capa PresentacionAutenticacion: Corresponde a la vista LoginView.vue del frontend Vue.js. Presenta el formulario de inicio de sesión y el formulario de registro al usuario. Captura las credenciales y las envía hacia la capa de negocio.

Capa NegocioAutenticacion: Compuesta por los componentes SecurityConfig, JwtAuthenticationConverter, AuthUtils y KeycloakAdminService. SecurityConfig define

las reglas de acceso por rol (ADMINISTRADOR, AGENDADOR, MEDICO_TERAPISTA, PACIENTE). JwtAuthenticationConverter extrae los roles del token JWT. KeycloakAdminService interactúa con la API de administración de Keycloak para el registro de nuevos usuarios.

Capa DatosAutenticacion: El registro de pacientes se persiste en la base de datos H2 mediante PacienteRepository (Spring Data JPA). Keycloak mantiene de forma independiente su propio almacén de credenciales. Esta capa accede al módulo de Servicios Transversales para la conexión a base de datos.

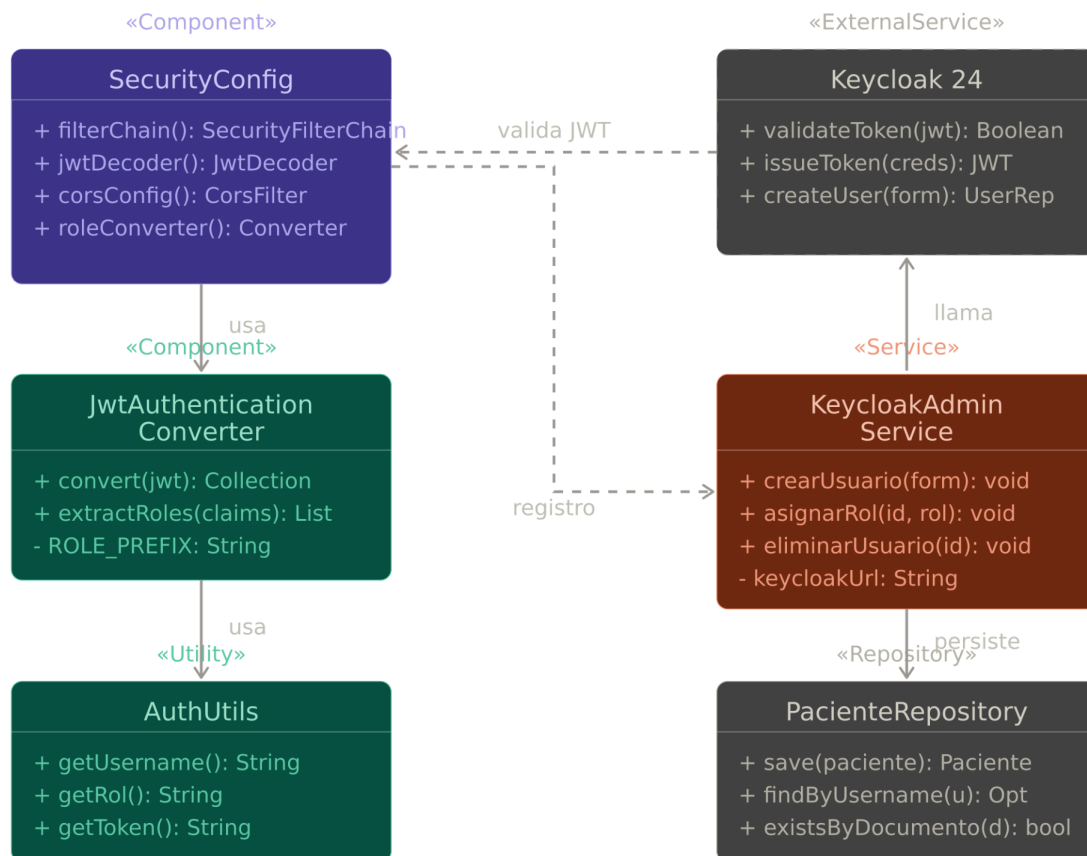


Figura X. Diagrama de clases simplificado — Módulo Autenticación.

5.1.2.2 Módulo 2: Agendamiento

El módulo de Agendamiento ha sido descompuesto siguiendo un estilo de capas. Es el núcleo funcional del sistema, pues centraliza la creación, reagendamiento, cancelación y consulta de citas médicas. La validación de disponibilidad ocurre tanto a nivel de servicio (excepción de negocio) como a nivel de base de datos (restricción unique), garantizando consistencia ante accesos concurrentes. Las capas identificadas son:

Capa PresentacionAgendamiento: Contiene a las vistas CrearCita.vue, ListarCitas.vue y PortalPaciente.vue del frontend. CrearCita.vue permite al Agendador registrar citas manualmente, mostrando las franjas horarias disponibles calculadas por el backend. ListarCitas.vue permite consultar y exportar citas por médico y fecha. PortalPaciente.vue ofrece al Paciente las mismas funcionalidades de forma autónoma tras autenticarse.

Capa NegocioAgendamiento: El componente CitaService centraliza toda la lógica de negocio. Valida la disponibilidad del médico consultando su configuración horaria (a través de ConfiguracionSistemaService) y calcula las franjas disponibles dentro de la ventana de agendamiento. Para registrar o reagendar una cita, CitaService interactúa

con `PacienteService` y `MedicoTerapistaService` para verificar que los actores involucrados existan y estén activos. `CitaController` expone los endpoints REST protegidos por rol.

Capa DatosAgendamiento: `CitaRepository` (Spring Data JPA) define las consultas necesarias para la persistencia de citas. Incluye consultas por médico y fecha, por paciente, y la restricción `unique` que previene la doble reserva. Esta capa se comunica con el módulo de Servicios Transversales para la conexión y ejecución de instrucciones sobre la base de datos H2.

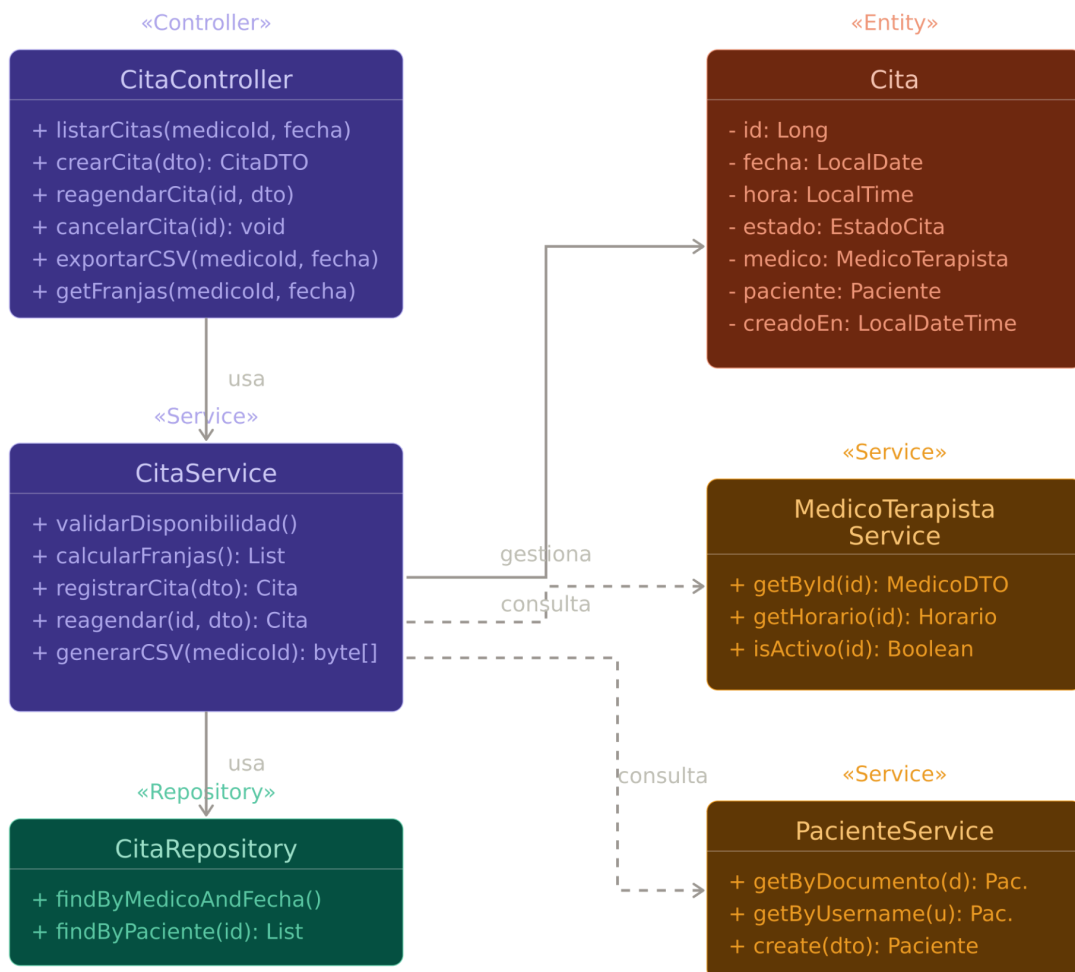


Figura Y. Diagrama de clases simplificado — Módulo Agendamiento.

5.2 Parte Dinámica (Comportamiento)

La parte dinámica (behavior) de la vista lógica es modelada a través del view type C&C, siguiendo la notación brindada en la guía y complementada con el trabajo presentado por [Roh04]. De acuerdo a esta perspectiva y de la aplicación del método ADD con los drivers identificados, los patrones que satisfacen la arquitectura y que pueden ser aplicados en conjunto son los siguientes:

- En la primera descomposición: Los componentes principales del sistema se estructuran a través de un esquema **Cliente/Servidor**. La interacción entre las unidades de negocio hace que ésta sea de tipo solicitud/respuesta.
- Para la segunda descomposición: En general, cada subsistema identificado incluye diferentes niveles de funcionalidad que será desarrollado, empaquetado e instalado en diferentes módulos, componentes y nodos. Estos niveles que en la vista estructural resultaron similares a las capas, coinciden en la vista dinámica con una dinámica de **n-tiers**.

5.2.1 View Packet Negocio Autenticación

Es una representación visual del flujo de autenticación y autorización del sistema PiedraAzul. La autenticación se delega completamente a Keycloak como servidor de identidad externo, mientras que Spring Security actúa como Resource Server OAuth2, validando los tokens JWT en cada petición entrante.

La vista permite comprender cómo los clientes obtienen acceso al sistema, cómo se validan sus identidades y cómo el backend verifica los permisos antes de procesar cada solicitud.

5.2.2 Vista de Componentes y Conectores

La vista de componentes y conectores del módulo de autenticación describe la interacción entre el cliente, Keycloak y el backend de Spring Boot. El conector principal es el protocolo OAuth2/JWT que permite la comunicación segura entre las unidades.

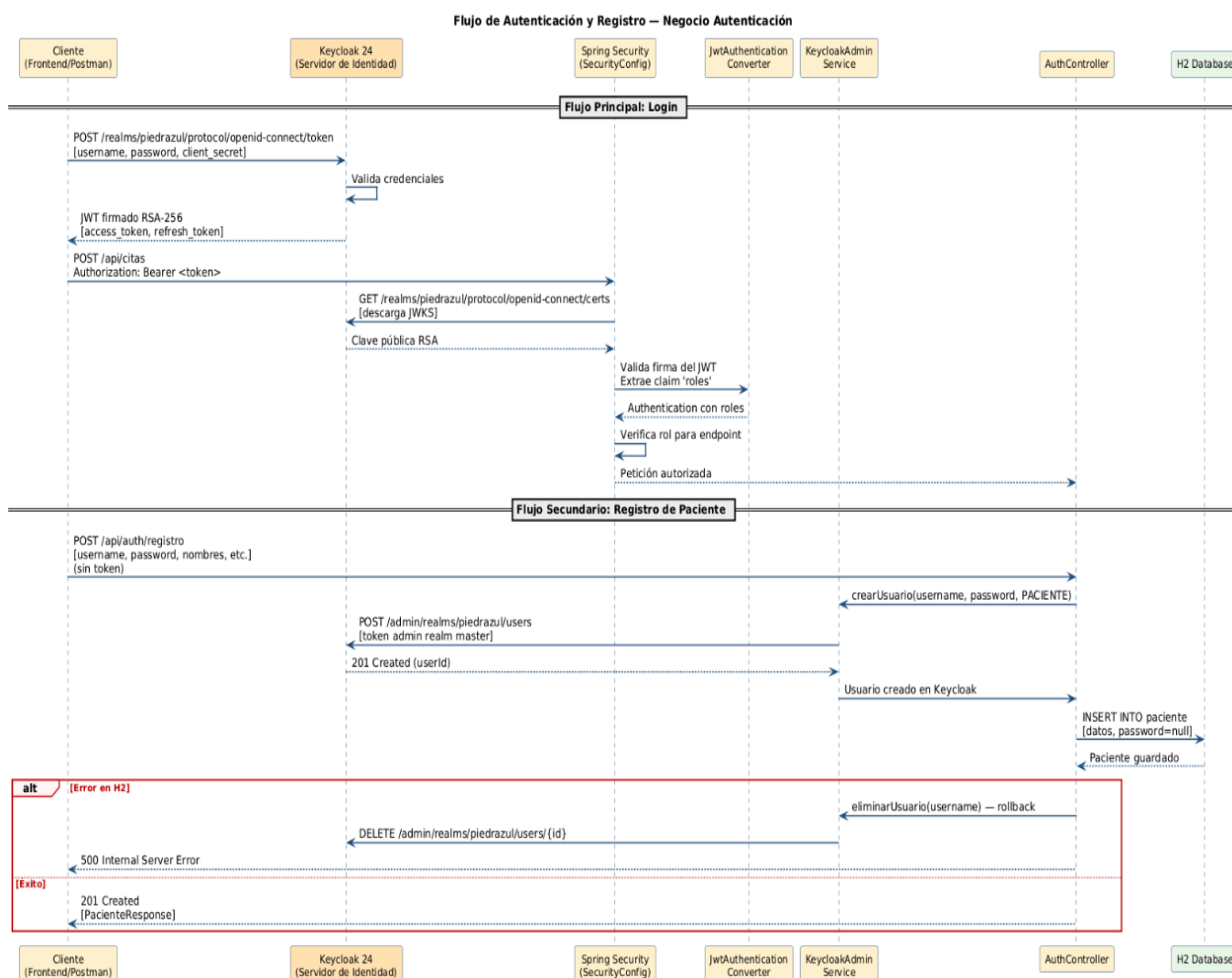


Figura 4. Vista de Componentes y Conectores — Negocio Autenticación

5.2.2.1 Catálogo de componentes

Nombre del View Packet	Negocio Autenticación
Rationale	La autenticación se delega completamente a Keycloak. El backend actúa como Resource Server OAuth2. El flujo Resource Owner Password Credentials permite el login desde el frontend sin redirección. Garantiza seguridad, trazabilidad de roles y rollback transaccional en el registro.
Componente	Keycloak 24 (externo), SecurityConfig, JwtAuthenticationConverter, AuthUtils, KeycloakAdminService
Descripción:	Componente encargado de la autenticación y de poder ingresar con las credenciales correspondientes al sistema
Composición Interna	<pre> graph TD subgraph VP [Viewpacket Negocio Autenticación] subgraph CP [Capa Presentación] F[Frontend Vue.js] P[Postman / API Client] end subgraph CS [Capa Seguridad] SC[SecurityConfig + JwtAuthConverter] KAS[KeycloakAdminService] end subgraph CN [Capa Negocio] AC[AuthController] AU[AuthUtils] end subgraph CD [Capa Datos / Identidad] H2[H2 Database Pacientes] K24[Keycloak 24 «externo»] end end F --> J1(()) P --> J1 J1 -- OAuth2/JWT --> SC J1 -- Admin API --> KAS SC --> J2(()) KAS --> J2 J2 -- Petición autorizada --> AC J2 --> J3(()) AC --> J3 J3 -- INSERT Paciente --> H2 J3 --> J4(()) AU --> J4 J4 -- CRUD Usuarios --> K24 J4 --> J5(()) K24 --> J5 K24 -.-> N1[password = null (autenticación via Keycloak)] K24 -.-> N2[Realm: piedrazul Rol: PACIENTE Protocolo: RSA-256] </pre>

5.2.3 View Packet Negocio Agendamiento

Es una representación visual de la estructura y el comportamiento del módulo de agendamiento de citas médicas. Esta vista proporciona una perspectiva de alto nivel de cómo el sistema valida disponibilidad, registra citas y calcula franjas horarias.

La vista de componentes y conectores permite identificar los componentes principales: CitaController, CitaService y CitaRepository, y cómo se agrupan para garantizar la consistencia de los datos de agendamiento.

5.2.3.1 Vista de Componentes y Conectores

La vista de componentes y conectores del módulo de agendamiento describe el flujo de creación de citas y consulta de franjas disponibles. El componente central es CitaService, que centraliza la validación de disponibilidad antes de persistir cualquier cita.

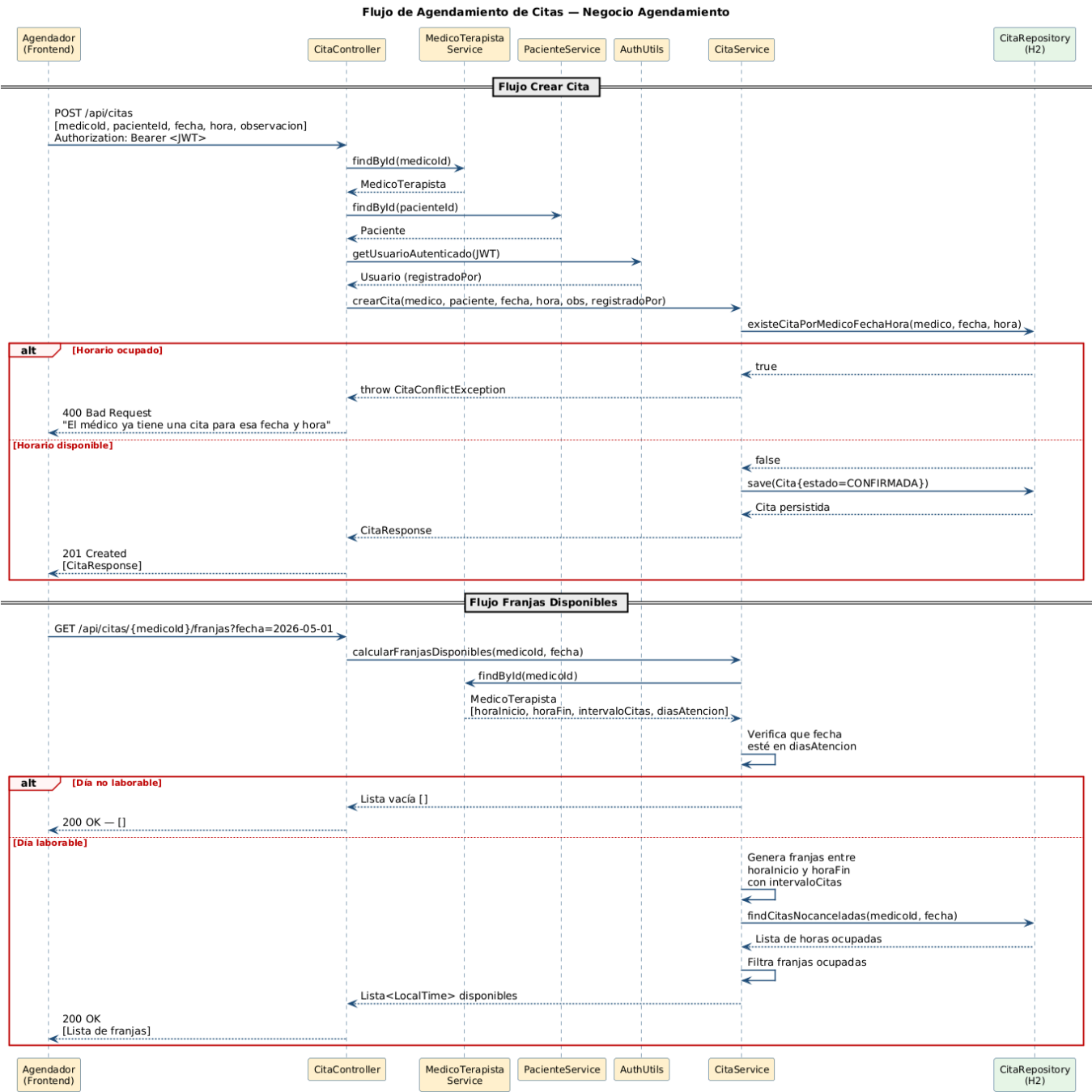


Figura 5. Vista de Componentes y Conectores — Negocio Agendamiento

5.2.3.2 Catálogo de componentes

Nombre del View Packet	Negocio Agendamiento
Rationale	El agendamiento centraliza la lógica de disponibilidad en CitaService. La validación ocurre tanto a nivel de servicio

	(excepción de negocio) como a nivel de base de datos (constraint unique), garantizando consistencia ante concurrencia. El acceso está protegido por rol mediante Spring Security.
Componente	CitaController, CitaService, CitaRepository, AuthUtils, MedicoTerapistaService, PacienteService
Descripción:	Componente encargado de la creación de citas
Composición Interna	<pre> graph TD subgraph Viewpacket_Negocio_Agendamiento subgraph Capa_Presentación Frontend[Frontend (Vue.js)] end subgraph Capa_Controlador CitaController[CitaController] end subgraph Capa_Servicio CitaService[CitaService] AuthUtils[AuthUtils] MedicoTerapistaService[MedicoTerapistaService] PacienteService[PacienteService] end subgraph Capa_Repositorio_Datos CitaRepository[CitaRepository] MedicoTerapistaRepository[MedicoTerapistaRepository] PacienteRepository[PacienteRepository] H2Database[H2 Database] end end Frontend -- "REST / JSON + JWT" --> CitaController CitaController -- "crearCita(), validarDisponibilidad()" --> CitaService CitaController -- "findById()" --> MedicoTerapistaService CitaController -- "findById()" --> PacienteService CitaService -- "save() / findBy()" --> CitaRepository MedicoTerapistaService --> MedicoTerapistaRepository PacienteService --> PacienteRepository CitaRepository --> H2Database MedicoTerapistaRepository --> H2Database PacienteRepository --> H2Database Note1[Regla de negocio: No puede haber dos citas para el mismo médico, fecha y hora] Note2[Extrae usuario autenticado desde JWT (registradoPor)] </pre>

5.2.4 View Packet Negocio Configuración

Es una representación visual del módulo de configuración del sistema, exclusivo del rol ADMINISTRADOR. Esta vista describe cómo se gestionan los parámetros operativos del sistema, en particular la configuración horaria de los médicos terapeutas.

El patrón Singleton a nivel de datos garantiza que exista un único registro de ConfiguracionSistema, evitando inconsistencias en los parámetros globales del sistema.

5.2.4.1 Vista de Componentes y Conectores

La vista de componentes y conectores del módulo de configuración describe el flujo de actualización de horarios de los médicos. El componente ConfiguracionSistemaService coordina la modificación y su impacto inmediato en el cálculo de franjas disponibles del módulo de agendamiento.

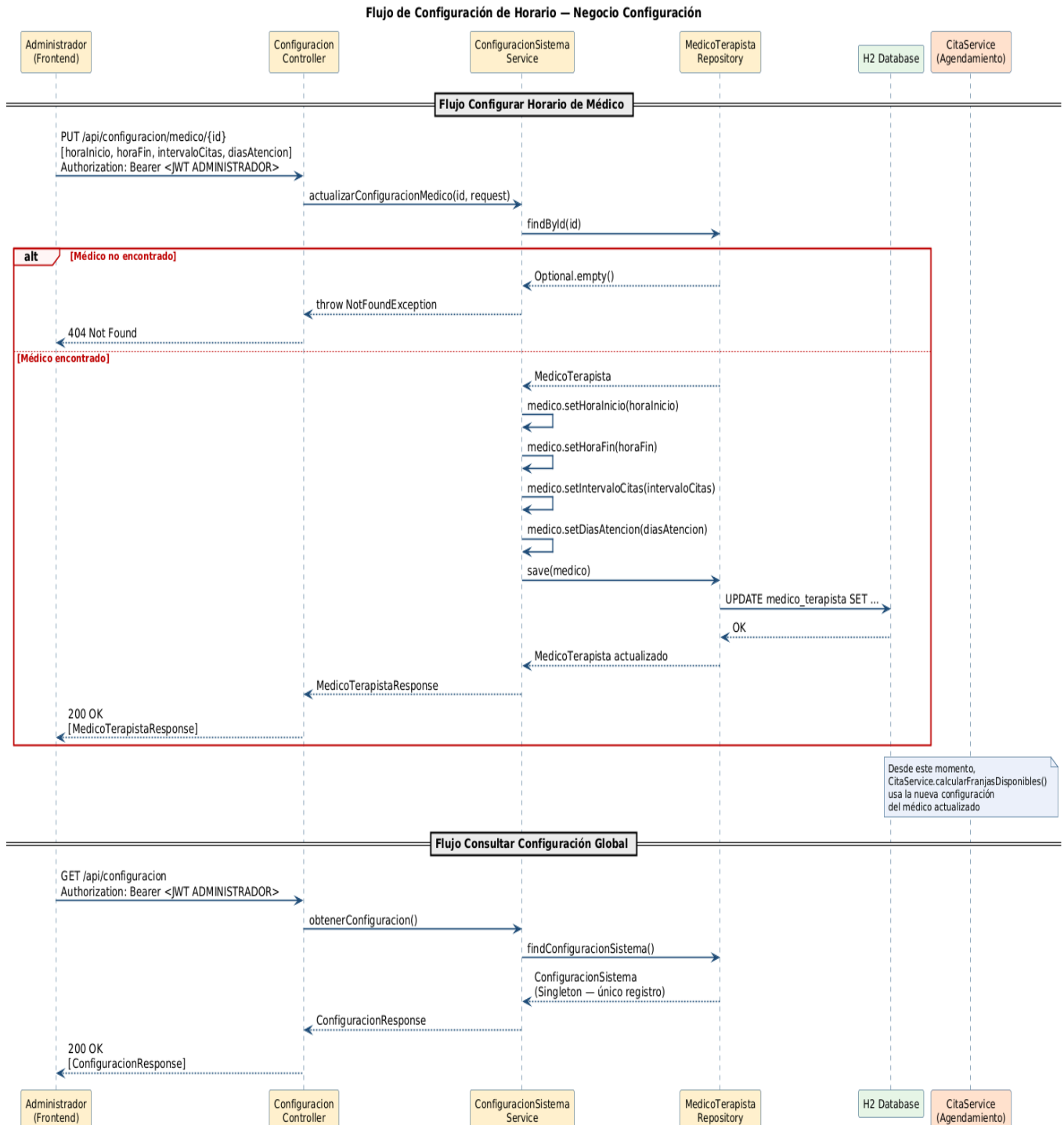


Figura 6. Vista de Componentes y Conectores — Negocio Configuración

5.2.4.2 Catálogo de componentes

Nombre del View Packet	Negocio Configuración
Rationale	La configuración es exclusiva del rol ADMINISTRADOR. Existe un único registro de ConfiguracionSistema (patrón Singleton a nivel de datos). La configuración de médicos se delega a MedicoTerapistaRepository. Los cambios en la configuración horaria impactan de inmediato en el cálculo de franjas disponibles del módulo de agendamiento.
Componente	ConfiguracionController, ConfiguracionSistemaService, ConfiguracionSistemaRepository, MedicoTerapistaRepository
Descripción:	Componente encargado de la configuración de parametros para el sistema
Composición Interna	<pre> graph TD subgraph VP [Viewpacket Negocio Configuración] subgraph CP [Capa Presentación] FA[Frontend Admin (Vue.js)] end subgraph CC [Capa Controlador] CC_Cont[ConfiguracionController] end subgraph CS [Capa Servicio] CSS[ConfiguracionSistemaService] end subgraph CRD [Capa Repositorio / Datos] CSSR[ConfiguracionSistemaRepository «Singleton»] MTR[MedicoTerapistaRepository] H2[H2 Database] end subgraph MI [Módulo Impactado] CS[CitaService (Agendamiento)] end end FA -- "REST / JSON + JWT (ADMIN)" --> CC_Cont CC_Cont -- "actualizarConfiguracion()
obtenerConfiguracion()" --> CSS CSS -- "findConfiguracionSistema()" --> CSSR CSS -.-> MTR MTR -.-> H2 CSSR -- "findById() save()" -.-> MTR CSSR --> H2 MTR --> H2 H2 --> CS CS -- "calcularFranjasDisponibles() usa nueva config" --> CS </pre> Patrón Singleton: Solo existe un registro de ConfiguracionSistema Los cambios de configuración impactan inmediatamente en el cálculo de franjas del módulo de Agendamiento

6. Vista de Despliegue

Componentes desplegados:

El sistema Piedrazul se compone de los siguientes nodos de despliegue:

- **Nodo 1 Cliente Web (Navegador):** El frontend desarrollado en Vue 3 + Vite se carga en el navegador del usuario final (Chrome, Firefox, Safari). No requiere instalación. Se comunica con el backend exclusivamente mediante peticiones HTTP/REST con JSON, adjuntando el token JWT en el header Authorization.
- **Nodo 2 Servidor de Aplicaciones (Spring Boot):** El backend se despliega como un único artefacto .jar ejecutado sobre una JVM (Java 21). Contiene todos los módulos del sistema (Autenticación, Agendamiento, Configuración y Servicios Transversales) empaquetados en una sola unidad. Se comunica con Keycloak para validar tokens JWT mediante el endpoint JWKS, y con la base de datos H2 mediante JPA/Hibernate. Expone sus endpoints REST en el puerto 8081.
- **Nodo 3 Servidor de Identidad:** Se despliega como contenedor Docker de forma independiente al servidor de aplicaciones. Gestiona el ciclo de vida completo de identidad: autenticación de usuarios, emisión de tokens JWT firmados con RSA-256, gestión de roles (ADMINISTRADOR, AGENDADOR, MEDICO_TERAPISTA, PACIENTE) y registro de nuevos pacientes mediante su API de administración. Opera en el puerto 8080.
- **Nodo 4 Base de datos (H2):** La base de datos H2 opera en modo embebido dentro del mismo proceso del servidor Spring Boot durante el entorno de desarrollo y pruebas académicas. Los datos se persisten en un archivo local. En un despliegue productivo, este nodo debería reemplazarse por un servidor PostgreSQL independiente para garantizar la durabilidad de los datos ante reinicios del servidor de aplicaciones.

6.1 Tecnología requerida

Sistema operativo: Este programa ha sido probado únicamente en sistema operativo linux y windows, y android sin embargo debería funcionar sin ningún problema en un sistema operativo macOS y iOS.

Lenguajes de programación: Se identifican los lenguajes de programación que se utilizaron para desarrollar el sistema, como Java, Vue, CSS, HTML, y JavaScript.

Frameworks y bibliotecas: Se utilizó el framework de Vue 3 y springboot.

Bases de datos: La base de datos necesaria para almacenar y gestionar es H2, la cual se puede cambiar en un futuro.

Protocolos y estándares: Los protocolos de comunicación y los estándares que se utilizan para la interoperabilidad entre componentes del sistema, son HTTP, REST, y JSON.

Herramientas de desarrollo: Durante el proceso de desarrollo, se utilizó Visual Studio, IntelliJ IDEA, y Git.

Servicios en la nube: El sistema no tiene servicios en la nube por el momento.

7. Rationale

Driver Candidato	Incumbencias	Tácticas	Patrones
Seguridad	Control de acceso por rol, protección de datos clínicos	Autenticación delegada, autorización por rol, tokens firmados RSA-256	Cliente-Servidor, Capas, Resource Server OAuth2
Usabilidad	Registro de cita en menos de 2 minutos, visualización de franjas disponibles	Separar interfaz de usuario de lógica de negocio, cálculo automático de franjas	Capas, MVC
Modificabilidad	Agregar nuevos módulos sin afectar los existentes	Encapsulamiento por dominio, interfaces bien definidas entre módulos	Monolito Modular, Capas

Tabla 3. Drivers, Tácticas y Patrones

7.1 Decisiones Arquitectónicas

- Se seleccionó la arquitectura de Monolito Modular como estilo principal porque permite mantener la simplicidad de un único artefacto desplegable (apropiado para el tamaño del equipo y el contexto académico del proyecto) sin renunciar a la organización interna por dominios de negocio. A diferencia de un monolito en capas tradicional, esta arquitectura garantiza que los cambios en el módulo de Configuración no afecten directamente al módulo de Agendamiento, reduciendo el riesgo de regresiones. Adicionalmente, si el sistema escala en el futuro, los módulos ya tienen los límites bien definidos para ser extraídos como microservicios independientes.
- Se seleccionó el patrón Cliente-Servidor entre el frontend Vue.js y el backend Spring Boot porque permite definir claramente el rol de quien expone los servicios (backend) y quien los consume (frontend), facilitando el desarrollo paralelo por parte del equipo. La comunicación exclusivamente mediante API REST con JSON garantiza el desacoplamiento entre ambas capas.
- Se delegó la autenticación completamente a Keycloak como servidor de identidad externo porque implementar un sistema de login propio desde cero implicaba construir y mantener manualmente el hashing de contraseñas, la emisión y validación de tokens JWT, la gestión de sesiones y las políticas de roles. Keycloak resuelve todos estos problemas con un estándar de industria (OAuth 2.0 / OIDC) y permite gestionar roles sin necesidad de deploy del backend. El backend actúa como Resource Server OAuth2, validando la firma RSA-256 del token en cada petición mediante el endpoint JWKS de Keycloak, sin almacenar credenciales propias.
- Se aplicó el patrón de Capas dentro de cada módulo (Presentación, Negocio/Servicio, Repositorio/Datos) para separar la lógica de presentación de la lógica de negocio y de la lógica de acceso a datos. Esto permite que un cambio en la estructura del DTO de respuesta no afecte la lógica de validación de disponibilidad de citas, y que un cambio en la base de datos (por ejemplo, migrar de H2 a PostgreSQL) no afecte los servicios ni los controladores.
- Se adoptó el patrón DTO con Mappers dedicados por módulo para garantizar que las entidades JPA nunca se expongan directamente al cliente. Esto protege campos sensibles de la base de datos y desacopla el contrato de la API REST de la estructura interna de persistencia, permitiendo que ambos evolucionen de forma independiente.

Modelo C4:  C4-Segundo Corte

URL Github: <https://github.com/Alexmax404/PIEDRA-AZUL---SWIII->

Video YouTube: <https://www.youtube.com/watch?v=Oss9PQotLnk>